Case Study

AI Tax Return Agent Prototype

Objective: Build an end-to-end prototype of an AI agent capable of handling basic tax return preparation tasks, including data input, tax calculations, and generating a simple tax return form.

1. Introduction and Overview

- **Problem**: Describe the need for automated tax return preparation, emphasizing how AI can streamline the process and increase accuracy.
- **Goal**: Create a working prototype of an AI agent that will automate basic tax return tasks such as collecting user input, performing tax calculations, and generating a tax return form.
- **Expectations**: Present a working prototype that includes:
    - Data collection from the user.
    - Tax calculation logic for a simple tax scenario.
    - Display of tax return details (e.g., tax owed, refund due).
    - A simple user interface (UI).

2. Requirements Gathering and Initial Setup

- **Technical Setup**:
    - Select a tech stack (e.g., Python, Flask for back-end, HTML/CSS for front-end).
    - Set up a development environment (e.g., Python with Flask for web framework, local server setup).
- **Prototype Requirements**:
    - **User Input**: Collect essential information like income, deductions, filing status.
    - **Tax Calculation**: Implement a basic tax calculator using known tax brackets or a simplified tax rule.
    - **Display Tax Results**: Show tax owed or refund due based on the user's input.

3. Building the User Input Interface

- **Form Creation**:
    - Design a simple frontend to collect essential information from the user (e.g., income, filing status, deductions) using lovable.dev, Cursor, or other AI tools to implement this.
    - Implement client-side validation to ensure input fields are filled out correctly.
- **Back-End Processing**:
    - Use Flask to handle form submissions and send the data to the server for processing.
    - Design routes that handle the input and display the next steps (e.g., tax calculations).
- **User Flow**:
    - On submission, redirect the user to a page that shows the tax result or an error message if input is incomplete.

4. Implementing the Tax Calculation Logic

- **Simple Tax Calculation**:
    - Use basic tax brackets to calculate the tax owed or refund due.
        - Example: Use a flat tax rate or a progressive tax system for simplicity.
        - Provide logic for deductions (e.g., standard deduction for a single filer).
- **Integration**:
    - Implement a Python function that performs the tax calculation based on the user's input.
    - Return the tax result to the front-end to be displayed in a clean, easy-to-understand format.
- **Testing**:
    - Run the calculations with sample data (*provide*) to verify correctness.
    - Ensure the results make sense for different scenarios (e.g., different income levels or filing statuses).

5. Displaying Tax Results and Final Form

- **Generate and Display Results**:
    - Show the tax owed or refund due on the results page after the user submits their information.
    - Use simple HTML templates to format the results for clarity.
- **Tax Return Form**:
    - Generate a simplified tax return form (e.g., a 1040 form or a mock form) with the calculated results displayed.
    - Allow the user to download or view the completed form.

6. Security Considerations **Data Security**:

    - Implement basic data validation and sanitization to prevent malicious inputs.
    - While full encryption and authentication may not be feasible in this prototype, emphasize the importance of secure data handling for a production version.
- **Compliance**:
    - Briefly discuss compliance considerations for a real-world tax system (e.g., handling user data, GDPR, and IRS e-filing requirements).

7. Final Touches and Presentation

- **User Interface Improvements**:
    - Polish the design with CSS to make the form and results page look professional and easy to navigate.
    - Ensure responsiveness for desktop and mobile viewing.
- **Documentation**:
    - Prepare a brief documentation of the system architecture, including how the tax calculation engine works, the data flow, and how the user interface interacts with the back-end.
- **Prepare the Demo**:
    - Walk through the system, from data input to tax calculation, and show how it generates the results.
    - Discuss the limitations of the prototype and possible improvements (e.g., adding real tax rules, integrating with IRS systems, or expanding the user interface).

8. Case Study Conclusion and Discussion

- **Review**:
    - Summarize the work done during the case study, emphasizing the challenges overcome and the learning process.
    - Discuss potential future developments for the AI tax agent prototype.
    - Address any questions or issues the team might have encountered.

Deliverables:

- A working prototype of an AI agent that can:
    - Collect user input (e.g., income, filing status).
    - Perform a simplified tax calculation.
    - Generate a basic tax return result page.
- Codebase with comments explaining key components.
- Documentation of the architecture and design decisions made.

Submission & Next Steps

- Please have your working prototype and code repository complete prior to your scheduled onsite meeting. This will allow us to review the Case Study during the interview.
- Note: Completion of this case study is a requirement to proceed with the internship interview process.

We appreciate the time and effort you are putting into this project and look forward to seeing your approach to the AI Tax Agent. Thank you for your continued interest in GreenGrowth CPAs.